# SENTINEL Lab 2 Brief: `libproc`

ECE 476 Engineering System Software • Fall 2026 • University of Victoria

| | |
|---|---|
| Lab session | Friday, October 30, 2026 |
| Prelab due | Wednesday, October 28, 2026, by midnight, on Brightspace. Required, with no marks[1] |
| Deliverable due | Thursday, November 12, 2026, by midnight, on Brightspace: 5% of the grade, out of 80 marks |
| Oral exam | In the Lab 3 session, Nov 20: 3% of the grade, out of 20 marks[2] |
| Lab 2 total | 8% of the course grade: the deliverable and the oral exam |

This brief is the one paper of Lab 2. It states what no other paper and no repository file states, and a footnote names the place of each other fact. Read it with `src/libproc/include/sentinel/proc.h`[3], which states each function: its steps in order, its errors and its log lines.

## 1 The task

Lab 2 builds `libproc`, the sensor network. Each of the four sensors runs as a process of its own, with a pipe to the centre. A named FIFO brings urgent contacts from other programs. A scheduler chooses the report that the centre handles next. Lab 2 does the work of two requirements: R1, four sensors that can stop,[4] and R2, an urgent contact that overtakes.[5]

Write the functions of these three files in `src/libproc/src/`, and one test (Section 5). Each file comes with a stub for each function, and its steps in a comment above it:

- `sensor.c`: `sensor_spawn`, `sensor_stop` and `sensor_reap`.
- `prio_channel.c`: `prio_open`, `prio_close` and `prio_send`. The file comes with `open_failed()` written: do not change it.
- `sched.c`: `sched_create`, `sched_destroy`, `sched_push` and `sched_pending`. The file comes with `struct sched` and `sched_next` written: do not change them.

Also change your Lab 1 code: add `O_CLOEXEC` to the flags of each `open()` of `sc_open_read`, `sc_open_write` and `sc_open_write_mode`, as `syscall.h` states.

The course gives the other files as source. Read them, and do not change them: `src/libproc/Makefile`, `proc.h`, and these files in `src/libproc/given/`:

- `channels.c` holds the table of sensors, and reads the channels.
- `centre.c` is the loop of the centre. Read it before you write `sensor.c`: it calls each of your functions.
- `assess.c` gives the class HIGH[6], and `policy.c` names the policies.
- `demo_lab2.c` is the demo.

## 2 Lab 2 in SENTINEL

Lab 2 makes three of the user stories[7] of the course true. It also makes true the case of R1 in which a sensor stops.

[1] The Lab 2 prelab, `docs/lab2/prelab2.pdf` in your repository. It is due before the Lab 2 session.

[2] The oral exam of Lab 2 is in the Lab 3 session. The Submission Guide, Section 3, `docs/submit/submit.pdf`, describes its format.

[3] `src/libproc/include/sentinel/proc.h`, in your repository. It states what each function of Lab 2 does, in order, with each of its errors and its log lines.

[4] R1 is a requirement of SENTINEL: four sensors run at very different rates, and a sensor can stop with no warning. The SENTINEL Lab Manual, Section 1.2, `docs/manual/sentinel_manual.pdf`, lists the six requirements.

[5] R2 is a requirement of SENTINEL: an urgent contact must overtake routine traffic. The SENTINEL Lab Manual, Section 1.2, `docs/manual/sentinel_manual.pdf`, lists the six requirements.

[6] HIGH is the priority that the centre gives: a contact that is on no list, in a region that the centre monitors. `P` is the flag of the sensor. The SENTINEL Lab Manual, Section 1.4, `docs/manual/sentinel_manual.pdf`, describes both.

[7] The user stories are in Section 5 of the ECE 476 Lab Documentation for SENTINEL, on Brightspace: seven states of the centre, from *i. Normal Operations* to *vii. Operator Decision.*

| Story | What Lab 2 makes true | Your code | What you see |
|---|---|---|---|
| i. Normal Operations | Four sensor processes run at the same time, each with its own pipe | `sensor_spawn` | Four children in `ps`, and four lines `sensor NAME started` in the log |
| ii. Critical Contact Detected | A report arrives on the channel of its sensor, so the channel names the sender | `sensor_spawn` | `contact ID from NWS` in the log. Lab 2 logs no position, so the page[8] does not show this state: `make lab1` shows it |
| iii. Threat Assessment | The centre gives the class HIGH[9], and the scheduler serves it first | `sched_next` | `served after ... ms, HIGH: unknown in STRAIT` in the log |
| A sensor stops | The centre collects it, writes an ALERT, and continues | `sensor_reap` | The ALERT line. While `make lab2 KILL=AUR MS=20000` runs, the page shows `Sensor health: DEGRADED` |

Figure 1 shows the processes and the channels of Lab 2, and who writes each part. Each sensor process runs the given program `contactfeed`.[10] The centre writes each report that it serves to the mission log.[11]

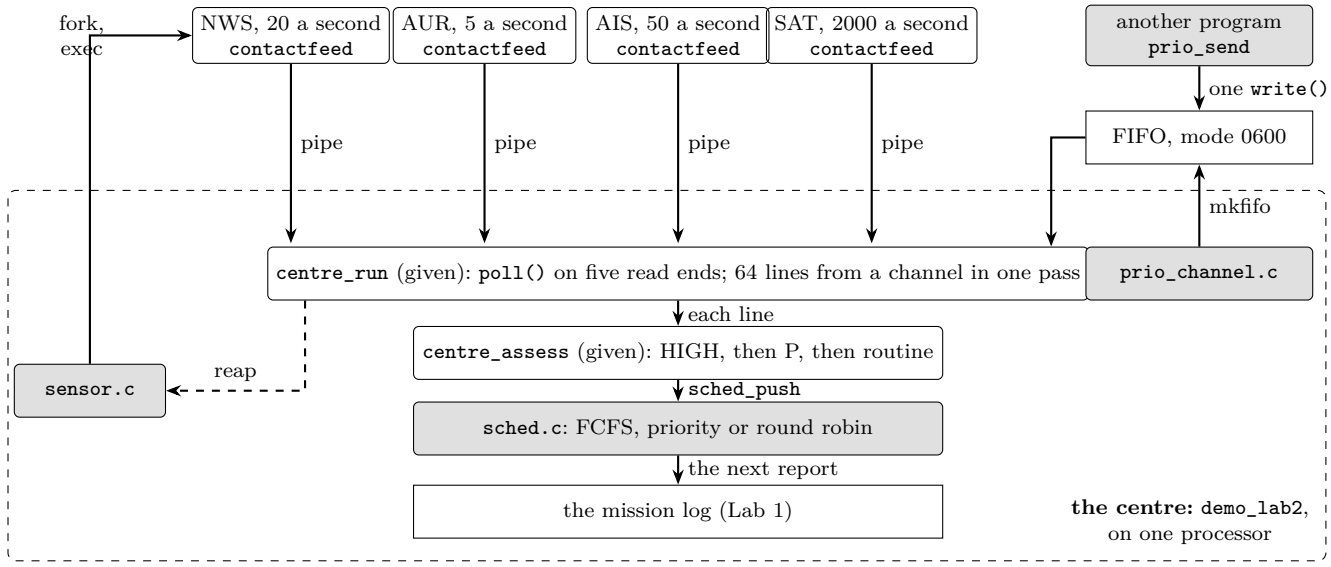



*Figure 1. The processes and the channels of Lab 2. Grey: your code. White: given code. Square corners: a file or a FIFO.*

# 3 The ideas

## 3.1 One call, two returns

`fork()` makes a copy of the process. Both processes continue after the call: `fork()` gives 0 to the copy, the child, and the pid of the child to the parent. The child gets a copy of the memory, and this includes the buffer of `stdout`. So flush `stdout` before `fork()`, or text that the parent printed can appear again from the child.

## 3.2 What exec keeps

`execvp()` replaces the program in a process with a new program. The process keeps its pid, its descriptors, its processors and its environment. It does not keep its memory. Each descriptor with the flag `O_CLOEXEC` closes at `exec`. Figure 2 shows the descriptors of the centre and of one sensor in `sensor_spawn`.

[8] The page is the operations centre display. Start it with `make watch`, and open `http://localhost:8080/`. `web/README.md`, in your repository, describes the page and each view.

[9] HIGH is the priority that the centre gives: a contact that is on no list, in a region that the centre monitors. `P` is the flag of the sensor. The SENTINEL Lab Manual, Section 1.4, `docs/manual/sentinel_manual.pdf`, describes both.

[10] `contactfeed` is the given program that writes contact reports in place of the four sensors. The SENTINEL Lab Manual, Section 3.1, `docs/manual/sentinel_manual.pdf`, describes it.

[11] The mission log is the file in which SENTINEL records each action, with its time (requirement R6). The Lab 1 Brief, Section 1, `docs/lab1/lab1_brief.pdf`, states what Lab 1 writes to it.

| fd | 1. The centre, after `pipe2()` | 2. The child, after `fork()` | 3. The child, after `dup2(7, 1)` and `execvp()` | 4. The centre, after `close(7)` |
|---|---|---|---|---|
| 0, 2 | the terminal | a copy | the terminal | the terminal |
| 1 | the terminal | a copy | **the write end of the pipe** | the terminal |
| 3 | the mission log[12], `O_CLOEXEC` | a copy | closed by `exec` | the mission log |
| 4, 5 | the FIFO: read end, write end, `O_CLOEXEC` | copies | closed by `exec` | the FIFO |
| 6 | the read end of the pipe, `O_CLOEXEC` | a copy | closed by `exec` | **the channel of the sensor** |
| 7 | the write end of the pipe, `O_CLOEXEC` | a copy | closed by `exec` | closed |

*Figure 2. The descriptors of the centre and of the NWS sensor in `sensor_spawn`. The numbers are those of a real run.*

`dup2(7, 1)` makes descriptor 1 a copy of the write end, and the copy has no `O_CLOEXEC`. So `contactfeed`[13] writes to standard output as usual, and its lines go into the pipe. Without `O_CLOEXEC` on descriptor 3, each sensor holds the mission log for its whole life. Without it on the pipes, each sensor holds the channels of the other sensors.

The centre restricts itself to one processor[14], and the child gets that restriction. So the child calls `cpu_unpin()` before `execvp()`. The environment also survives `exec`: that is how `SENTINEL_PROFILE` selects the profile[15] of each sensor.

## 3.3 A pipe ends when its last writer closes

A `read()` from a pipe gives 0, end of file, only when no process holds a write end. After `fork()`, the centre and the child each hold one. So the centre closes its copy at once. If it does not, the channel never ends, and the centre never sees that its sensor stopped.

## 3.4 Collect each child

A child that stops stays as a *zombie* until its parent collects it with `waitpid()`: the kernel keeps its exit status until then. With `WNOHANG`, `waitpid()` does not wait: it gives 0 when the child still runs. The status tells how the child stopped: `WIFEXITED` and `WEXITSTATUS` give an exit code, and `WIFSIGNALED` and `WTERMSIG` give a signal.

## 3.5 A channel with a name

`mkfifo()` makes a FIFO: a pipe with a name in the file system. Any program on the machine can open it, so the centre does not start its writers. Two facts change the code:

- `open()` of a FIFO waits until the other side opens it. The centre opens its read end with `O_NONBLOCK`, so that it does not wait.
- With no writer, a `read()` gives end of file at once, and `poll()` always marks the FIFO. So the centre holds a write end itself, and an empty FIFO is only empty.

Many programs can write to the FIFO. Linux writes `PIPE_BUF` (4096) bytes or fewer in one atomic step. A report has 128 bytes or fewer, so the lines of two writers cannot mix, if each line is one `write()`.

## 3.6 Who goes next

The scheduler chooses the next report among the reports that wait. It is not the Linux scheduler, which chooses among processes. A policy has a choice only when reports wait: with no queue, the three policies give the same order. Eight reports wait in this order when the centre takes the next one:

[12]The mission log is the file in which SENTINEL records each action, with its time (requirement R6). The Lab 1 Brief, Section 1, `docs/lab1/lab1_brief.pdf`, states what Lab 1 writes to it.

[13]`contactfeed` is the given program that writes contact reports in place of the four sensors. The SENTINEL Lab Manual, Section 3.1, `docs/manual/sentinel_manual.pdf`, describes it.

[14]The centre runs on one processor. `cpu_pin` and `cpu_unpin`, in `common/cpu.h`, set the processors that a process can use. The SENTINEL Lab Manual, Section 1.5, `docs/manual/sentinel_manual.pdf`, explains why.

[15]A profile selects the traffic that `contactfeed` sends: `world`, `ideal`, `dirty`, `field` or `threat`. The SENTINEL Lab Manual, Section 3.1, `docs/manual/sentinel_manual.pdf`, describes each one.

| **Arrival** | S1 | S2 | N1 | S3 | A1 | S4 | U1 | S5 |
|---|---|---|---|---|---|---|---|---|
| **Sensor, class**[16] | SAT | SAT | NWS, P | SAT | AIS | SAT, HIGH | AUR | SAT |
| **FCFS: the oldest** | S1 | S2 | N1 | S3 | A1 | S4 | U1 | S5 |
| **Priority: HIGH, P, routine** | S4 | N1 | S1 | S2 | S3 | A1 | U1 | S5 |
| **Round robin: NWS, AUR, AIS, SAT** | N1 | U1 | A1 | S1 | S2 | S3 | S4 | S5 |

Priority serves the HIGH report first, as R2 requires.[17] While urgent reports continue to arrive, the routine reports wait: that is *starvation*. Round robin serves the one report of AUR second, but the HIGH report seventh. Give the mean and the worst case of each wait: one number cannot show both a short usual wait and a long rare one.

# 4 The code rules

The code rules of the Lab 1 Brief[18] apply to Lab 2 too. `proc.h`[19] gives the steps of each function. The marks of the code rules (Section 7) check these steps. One rule of Lab 1 changes: a call that a step marks *No check* needs no branch and no comment. The step gives the reason.

# 5 Build, run and check

Lab 2 adds these targets to those of the SENTINEL Lab Manual.[20] Run each one at the top of your repository.

| **Command** | **What it does** |
|---|---|
| `make` | Builds `src/libproc` with the given `Makefile`. Until your three files exist, it compiles the files that exist, names the others, and builds no library |
| `make lab2` | Runs `demo_lab2` with your library for 3 s, for the page[21]. `MS=10000` runs it for 10 s, and `KILL=AUR` kills AUR after 0.5 s |
| `make accept2` | Runs the 26 checks of `tools/accept/accept_lab2.c` on your library and the demo |
| `make submit LAB=2` | Writes the zip[22] of Lab 2 (Section 6) |

`demo_lab2` states its options and its exit status in its first comment. This is the end of the output of `make accept2` when each check passes:

```
  sensor.c        13 of 13  100%
  prio_channel.c   6 of 6   100%
  sched.c          5 of 5   100%
  libsyscall       1 of 1   100%
  demo_lab2        1 of 1   100%

26 of 26 checks pass (100%).
```

A failed check prints a sentence about the contract. Read the check in `accept_lab2.c` to see what it does. Do not change the file: the marks come from the course copy of it.

**Your test.** `src/libproc/tests/test_lab2.c` comes with `read_lines()` and its first case written: a sensor with `burst` 5 gives 5 lines on its channel. A comment above each other test function states what to check: a second `sensor_spawn` gives `EEXIST`, `sensor_reap` collects a sensor that finished, and `prio_open` makes a FIFO of mode

[16] HIGH is the priority that the centre gives: a contact that is on no list, in a region that the centre monitors. `P` is the flag of the sensor. The SENTINEL Lab Manual, Section 1.4, `docs/manual/sentinel_manual.pdf`, describes both.

[17] R2 is a requirement of SENTINEL: an urgent contact must overtake routine traffic. The SENTINEL Lab Manual, Section 1.2, `docs/manual/sentinel_manual.pdf`, lists the six requirements.

[18] The rules of the code of Lab 1, which apply to each later lab: the Lab 1 Brief, Section 2, `docs/lab1/lab1_brief.pdf`.

[19] `src/libproc/include/sentinel/proc.h`, in your repository. It states what each function of Lab 2 does, in order, with each of its errors and its log lines.

[20] The make targets of all labs, such as `make test` and `make watch`: the SENTINEL Lab Manual, Section 4, `docs/manual/sentinel_manual.pdf`.

[21] The page is the operations centre display. Start it with `make watch`, and open `http://localhost:8080/`. `web/README.md`, in your repository, describes the page and each view.

[22] `make submit` builds your work from clean, tests it, and writes the zip. The Submission Guide, Section 5, `docs/submit/submit.pdf`, describes it, and Section 6 the upload.

0600 that `prio_close` removes. Write their bodies with `assert()`, as the Lab 1 Brief[23] describes. The marker reads each case, and runs the test. The course also gives `tests/test_policies.c`, which tests the eight reports of Section 3.6 under each policy with your `sched_create` and `sched_push`. Do not change it.

`make test LAB=2`[24] builds and runs both, with the tests of Lab 1. A failure of the kernel, such as a failed `fork()`, needs no test.

**valgrind.** This command checks the centre only, and exits with status 1 when it finds an error or a leak:

```
valgrind --leak-check=full --child-silent-after-fork=yes --error-exitcode=1 build/demo_lab2
```

It does not report the helper process of the demo, a copy of the centre that holds its memory, or the sensor processes, which run the given `contactfeed`.[25]

# 6 Hand it in

The hand-in of Lab 2 is your code and three files of your own runs, in `docs/lab2/`. `make submit LAB=2` puts them in the zip. Make each file with your own library.

## 6.1 The record run, and the capture

In terminal 1, at the top of your repository, run the demo for 20 s. It kills AUR after 0.5 s:

```
rm -f docs/lab2/missions.log
SENTINEL_PROFILE=world build/demo_lab2 --sched priority --ms 20000 --kill-sensor AUR \
    --expected common/traffic.txt --log docs/lab2/missions.log > docs/lab2/demo_output.txt
```

While it runs, and 2 s or more after its start, capture the processes and their descriptors in terminal 2: the centre, and its children that run `contactfeed`.

```
c=$(pgrep -o demo_lab2); s=$(pgrep -P $c contactfeed | head -1)
{ ps -o pid,ppid,stat,args --ppid $c -p $c; echo; ls -l /proc/$c/fd; echo;
  ls -l /proc/$s/fd; } > docs/lab2/proc_capture.txt
```

This is a part of `proc_capture.txt` from one run, with the paths shortened:

```
    PID    PPID STAT COMMAND
  24876   24873 R    build/demo_lab2 --sched priority --ms 20000 --kill-sensor AUR ...
  24878   24876 S    .../build/contactfeed --sensor NWS --rate 20
  24880   24876 S    .../build/contactfeed --sensor AIS --rate 50
  24881   24876 S    .../build/contactfeed --sensor SAT --rate 2000
  24882   24876 Z    [demo_lab2] <defunct>
l-wx------ 1 user user 64 Oct 12 07:17 3 -> .../docs/lab2/missions.log
lr-x------ 1 user user 64 Oct 12 07:17 6 -> pipe:[76681]
l-wx------ 1 user user 64 Oct 12 07:17 1 -> pipe:[76681]
```

Three sensors run: AUR stopped. The `Z` line is the helper process of the demo, which sent the urgent contact and stopped. It is a zombie until the demo collects it at the end of the run. Descriptor 1 of the NWS sensor and descriptor 6 of the centre are the same pipe, 76681. The sensor has no descriptor 3. Descriptors 0 and 2 are the terminal only when you start the demo in a terminal. If not, they show where its standard input and standard error go. The marker does not check them.

## 6.2 The checks of agreement

The marker compares the three files, and your code:

- The pid of each `sensor NAME started pid=...` line is in the capture, except the pid of AUR. The pid of AUR is in its ALERT line.

[23] Your own tests are C programs with `assert()` and no framework. The Lab 1 Brief, Section 4.2, `docs/lab1/lab1_brief.pdf`, states their rules.

[24] The make targets of all labs, such as `make test` and `make watch`: the SENTINEL Lab Manual, Section 4, `docs/manual/sentinel_manual.pdf`.

[25] `contactfeed` is the given program that writes contact reports in place of the four sensors. The SENTINEL Lab Manual, Section 3.1, `docs/manual/sentinel_manual.pdf`, describes it.

- The `fd=` of each sensor that runs is a channel of the centre in the capture. Descriptor 1 of the sensor is the same pipe.
- The ALERT for AUR comes before `Part 2 complete` in the log.
- The log shows that the centre served `ACCF-URGENT`, and `demo_output.txt` says `received`.

Then run `make submit LAB=2`, and upload the zip.[26]

# 7 Done when, and the marks

☐ `make` prints no warning, and `make test LAB=2`[27] passes your test and `test_policies`.
☐ `make accept2` gives 26 of 26.
☐ The record run exits with status 0, and each requirement line says `yes`.
☐ After a run, `ps` shows no `contactfeed`[28] process, and `/tmp/sentinel_priority.fifo` does not exist.
☐ `valgrind` reports no error and no leak.
☐ The three files of Section 6 are in `docs/lab2/`.
☐ The conditions of a finished Lab 1 in the Lab 1 Brief[29] still hold.

| Part | Marks |
|---|---|
| Acceptance test (`make accept2`), in proportion to the checks that pass | 20 |
| Course tests: the tests of the course, on each function of `proc.h`[30] that you write | 24 |
| Your test (Section 5) | 12 |
| Code rules, and the errors of each call (Section 4) | 12 |
| `valgrind`: no error and no leak; no zombie; the FIFO removed | 4 |
| Agreement of the hand-in files (Section 6.2) | 8 |
| **Total** | **80** |

You do not get the course tests. They check only what `proc.h` and this brief state. The oral exam has its own 20 marks.

[26] `make submit` builds your work from clean, tests it, and writes the zip. The Submission Guide, Section 5, `docs/submit/submit.pdf`, describes it, and Section 6 the upload.

[27] The make targets of all labs, such as `make test` and `make watch`: the SENTINEL Lab Manual, Section 4, `docs/manual/sentinel_manual.pdf`.

[28] `contactfeed` is the given program that writes contact reports in place of the four sensors. The SENTINEL Lab Manual, Section 3.1, `docs/manual/sentinel_manual.pdf`, describes it.

[29] The conditions of a finished Lab 1, such as no warning and a clean `valgrind`: the Lab 1 Brief, Section 5, `docs/lab1/lab1_brief.pdf`.

[30] `src/libproc/include/sentinel/proc.h`, in your repository. It states what each function of Lab 2 does, in order, with each of its errors and its log lines.